

UNIVERSIDAD PERUANA UNIÓN
FACULTAD DE INGENIERIA Y ARQUITECTURA
E. P. ING SISTEMAS



Desarrollo de Aplicaciones Distribuidas
Examen Unidad 2 - Sistema de Gestión de Talleres

Autor:

Miranda Gamarra Jeanpierre Samuel

Docente:

Mg. Benjamin David Reyna Barreto

Lima, Junio de 2026

Repositorio: <https://github.com/JeanpierreSam/DAD-examen-u2>

TABLA DE ENTREGABLES

#	Entregable	Estado
1	Código fuente de cada microservicio	✓ Incluido en repositorio
2	Documentación de API (OpenAPI/Swagger)	✓ Disponible por servicio y vía Gateway
3	Diagramas de arquitectura y despliegue	✓ Detallado en este documento
4	Configuraciones (dev, test, prod)	✓ Config Server + config-repo/
5	Scripts Dockerfile y docker-compose	✓ Multi-stage build por servicio
6	Pruebas unitarias e integración	✓ JUnit + Postman E2E (41 pruebas)
7	Guía de instalación y ejecución	✓ Docker Compose y ejecución local

1. INTRODUCCIÓN

El presente proyecto corresponde al desarrollo de una solución basada en **arquitectura de microservicios** para la gestión de talleres académicos. La implementación integra 7 servicios independientes organizados en dos capas:

- **Capa de Infraestructura:** Config Server, Eureka Server, API Gateway
- **Capa de Negocio:** ms-auth, ms-gestion-instructor, ms-gestion-alumno, ms-gestion-taller

Características Principales

- 7 microservicios independientes y escalables con Spring Boot 3.5.14 / Java 21
- Autenticación centralizada con JWT y control de acceso por roles (ADMIN, INSTRUCTOR, ALUMNO)
- Service Discovery con Netflix Eureka
- Configuración centralizada con Spring Cloud Config Server (perfil `native`)
- API Gateway con filtro global de autenticación y autorización
- Comunicación entre servicios vía OpenFeign con Circuit Breaker (Resilience4j)
- Patrón Saga con compensación para matrículas de talleres
- Documentación API con OpenAPI/Swagger
- Containerización completa con Docker y Docker Compose

2. CÓDIGO FUENTE DE CADA MICROSERVICIO

El repositorio incluye el código fuente completo de los 7 microservicios. Cada servicio sigue una estructura hexagonal con sus propias capas:

Estructura del Repositorio

```
examen-u2/
├── docker-compose.yml                # Orquestación de contenedores
├── DAD_Verificacion_E2E.postman_collection.json # Pruebas E2E automatizadas
├── config-repo/                      # Configuraciones centralizadas
│   ├── application.properties        # Config compartida
│   ├── ms-admin-api-gateway.properties
│   ├── ms-auth.properties
│   ├── ms-gestion-alumno.properties
│   ├── ms-gestion-instructor.properties
│   └── ms-gestion-taller.properties
├── config-server/                   # 1. Config Server (puerto 8888)
├── eureka-server/                   # 2. Eureka Server (puerto 8761)
├── api-gateway/                     # 3. API Gateway (puerto 8080)
├── ms-auth/                         # 4. Auth Service (puerto 8084)
├── ms-gestion-instructor/           # 5. Instructores (puerto 8081)
├── ms-gestion-alumno/               # 6. Alumnos (puerto 8082)
└── ms-gestion-taller/               # 7. Talleres (puerto 8083)
```

Responsabilidades por Microservicio

Microservicio	Patrón	Comunicación	Persistencia	Responsabilidad
ms-admin-config-server	Monolítico (Config Server)	HTTP (Clientes Config)	Git local	Centraliza la configuración externa
ms-admin-registry-server	Monolítico (Service Registry)	HTTP (Eureka)	En memoria	Registro y descubrimiento de servicios
ms-admin-api-gateway	Gateway (BFF)	HTTP (Clientes / Servicios)	No requiere	Punto único de entrada y enrutamiento
ms-auth	Microservicio (Hexagonal)	HTTP/gRPC (OpenFeign)	PostgreSQL (authdb)	Autenticación JWT y gestión de usuarios

ms-gestion-instructor	Microservicio (Hexagonal)	HTTP/gRPC (OpenFeign)	PostgreSQL (instructor_db)	Gestión de instructores
ms-gestion-alumno	Microservicio (Hexagonal)	HTTP/gRPC (OpenFeign)	PostgreSQL (alumno_db)	Gestión de alumnos
ms-gestion-taller	Microservicio (Hexagonal)	HTTP/gRPC (OpenFeign)	PostgreSQL (taller_db)	Gestión de talleres (compuesto)

Stack Tecnológico

Tecnología	Versión	Uso
Java	21	Lenguaje principal
Spring Boot	3.5.14	Framework base
Spring Cloud	2025.0.2	Ecosistema de microservicios
Spring Cloud Config Server	—	Configuración centralizada
Netflix Eureka	—	Service Discovery
Spring Cloud Gateway (MVC)	—	API Gateway
Spring Cloud OpenFeign	—	Comunicación entre servicios
Resilience4j	—	Circuit Breaker
Spring Data JPA / Hibernate	—	Persistencia ORM
Spring Security + JWT 0.12.6	—	Autenticación JWT
Springdoc OpenAPI	2.8.9	Documentación Swagger
PostgreSQL	16 Alpine	Base de datos relacional
Docker / Docker Compose	3.8	Containerización y orquestación
Maven	—	Gestión de dependencias

3. DOCUMENTACIÓN DE API (OPENAPI/SWAGGER)

Todos los microservicios de negocio incluyen documentación automática con **Springdoc OpenAPI**.

URLs de Swagger UI por Servicio

Servicio	Swagger UI	OpenAPI JSON
ms-auth	http://localhost:8084/swagger-ui.html	http://localhost:8084/api-docs
ms-gestion-instructor	http://localhost:8081/swagger-ui.html	http://localhost:8081/api-docs
ms-gestion-alumno	http://localhost:8082/swagger-ui.html	http://localhost:8082/api-docs

ms-gestion-taller	http://localhost:8083/swagger-ui.html	http://localhost:8083/api-docs
-------------------	---------------------------------------	--------------------------------

Acceso vía Gateway (Puerto 8080)

Servicio	URL via Gateway
Instructores	http://localhost:8080/api/instructores/swagger-ui.html
Alumnos	http://localhost:8080/api/alumnos/swagger-ui.html
Talleres	http://localhost:8080/api/talleres/swagger-ui.html
Auth	http://localhost:8080/api/auth/swagger-ui.html

Endpoints Principales

Autenticación (/api/auth)

Método	Endpoint	Descripción	Rol
POST	`/api/auth/register`	Registrar nuevo usuario	Público
POST	`/api/auth/login`	Autenticar y obtener JWT	Público
GET	`/api/auth/validate`	Validar token JWT	Autenticado

Instructores (/api/instructores)

Método	Endpoint	Descripción	Rol
GET	`/api/instructores`	Listar todos	Autenticado
GET	`/api/instructores/{id}`	Obtener por ID	Autenticado
POST	`/api/instructores`	Crear instructor	ADMIN
PUT	`/api/instructores/{id}`	Actualizar instructor	ADMIN
DELETE	`/api/instructores/{id}`	Eliminar instructor	ADMIN

Alumnos (/api/alumnos)

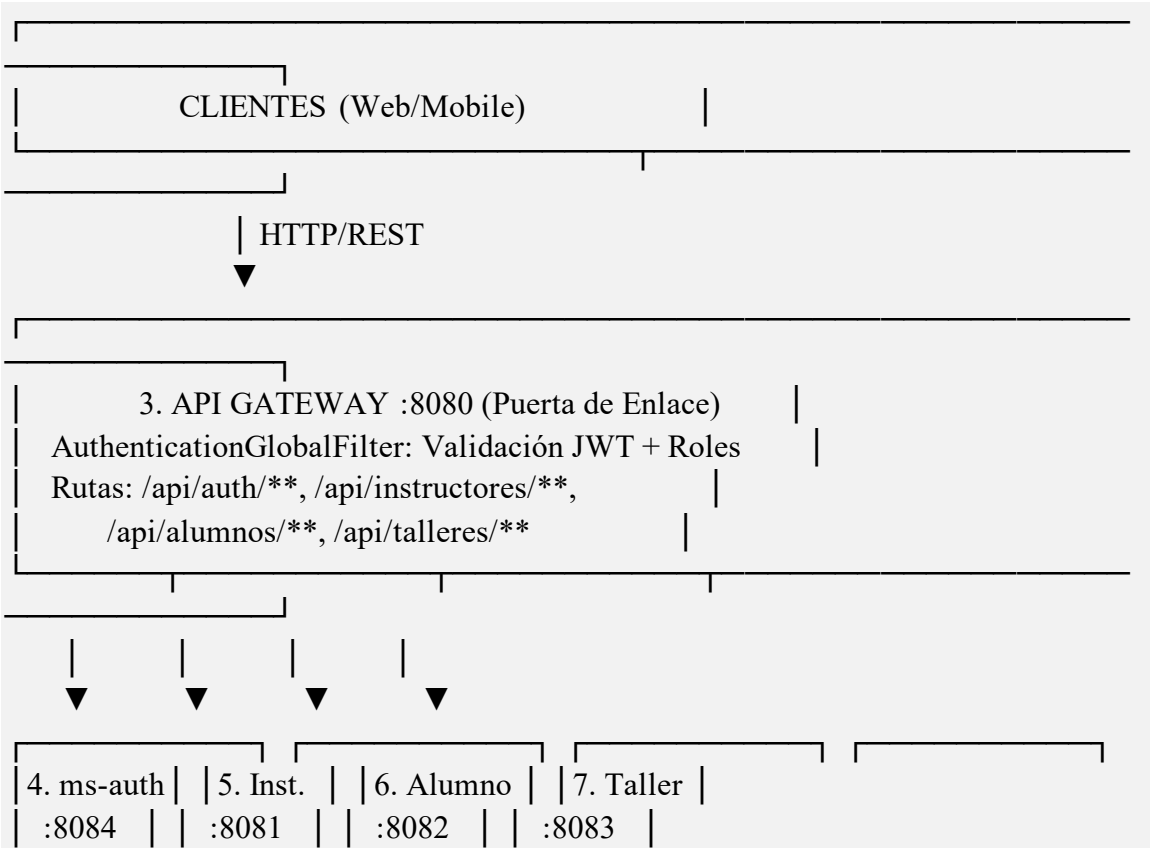
Método	Endpoint	Descripción	Rol
GET	`/api/alumnos`	Listar todos	Autenticado
GET	`/api/alumnos/{id}`	Obtener por ID	Autenticado
POST	`/api/alumnos`	Crear alumno	ADMIN
PUT	`/api/alumnos/{id}`	Actualizar alumno	ADMIN
DELETE	`/api/alumnos/{id}`	Eliminar alumno	ADMIN
POST	`/api/alumnos/{id}/incrementar-taller`	Incrementar contador talleres	ADMIN
POST	`/api/alumnos/{id}/decrementar-taller`	Decrementar contador talleres	ADMIN

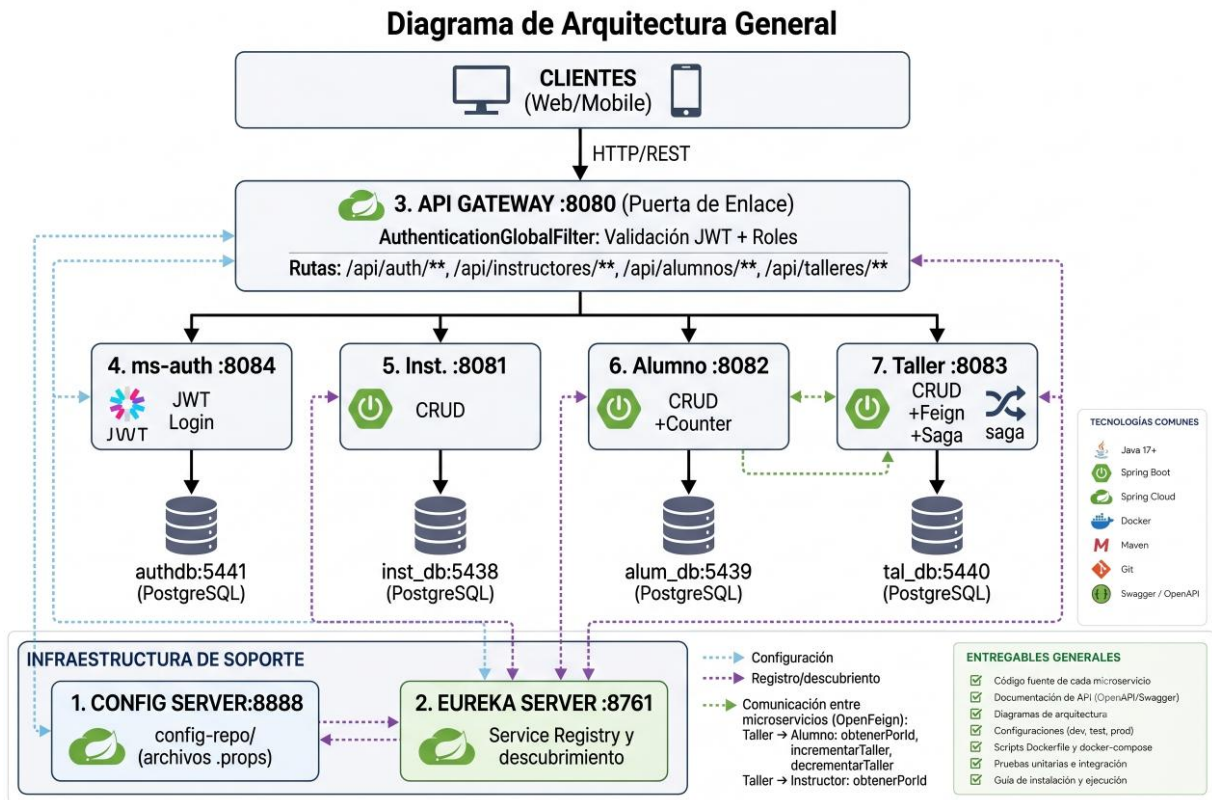
Talleres (/api/talleres)

Método	Endpoint	Descripción	Rol
GET	`/api/talleres`	Listar todos	Autenticado
GET	`/api/talleres/{id}`	Obtener por ID	Autenticado
POST	`/api/talleres`	Crear taller	ADMIN
PUT	`/api/talleres/{id}`	Actualizar taller	ADMIN
DELETE	`/api/talleres/{id}`	Eliminar taller	ADMIN
POST	`/api/talleres/{tallerId}/asignar-instructor/{instructorId}`	Asignar instructor	ADMIN
POST	`/api/talleres/{tallerId}/inscribir-alumno/{alumnoId}`	Inscribir alumno	ALUMNO
POST	`/api/talleres/{tallerId}/matricular-alumno/{alumnoId}`	Matricular con saga	ALUMNO
GET	`/api/talleres/{id}/detalle-completo`	Detalle completo (Feign)	Autenticado

4. DIAGRAMAS DE ARQUITECTURA Y DESPLIEGUE

Diagrama de Arquitectura General





Puertos y Bases de Datos

Servicio	Puerto	Base de Datos	Puerto DB
config-server	8888	— (nativa)	—
eureka-server	8761	— (memoria)	—
api-gateway	8080	—	—
ms-auth	8084	authdb (PostgreSQL)	5441
ms-gestion-instructor	8081	instructor_db (PostgreSQL)	5438
ms-gestion-alumno	8082	alumno_db (PostgreSQL)	5439
ms-gestion-taller	8083	taller_db (PostgreSQL)	5440

Patrones de Diseño Implementados

Patrón	Implementación	Responsabilidad
API Gateway	Spring Cloud Gateway MVC	Punto único de entrada, JWT, CORS
Service Discovery	Netflix Eureka Server	Registro dinámico, health checks
Externalized Config	Spring Cloud Config Server	Configuración centralizada en config-repo/

Circuit Breaker	Resilience4j	Tolerancia a fallos en llamadas Feign
Saga con Compensación	TallerServiceImpl.matricularAlumno	Matrícula con rollback automático
Backend for Frontend (BFF)	API Gateway	Rutas específicas por dominio
Database per Service	4 PostgreSQL independientes	Aislamiento y escalabilidad

5. CONFIGURACIONES (DEV, TEST, PROD)

El proyecto usa **Spring Cloud Config Server** con perfil native, leyendo configuraciones desde config-repo/.

Configuración Compartida (`application.properties`)

```
# Eureka
eureka.client.service-
url.defaultZone=${EUREKA_DEFAULT_ZONE:http://localhost:8761/eureka}

# Actuator
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=always

# JWT Secret (compartido entre Auth y Gateway)
jwt.secret=dad-upeu-microservicios-jwt-secret-key-2026-super-segura-0123456789
```

Configuración ms-auth (`ms-auth.properties`)

```
server.port=8084
spring.application.name=ms-auth
spring.datasource.url=jdbc:postgresql://localhost:5441/authdb
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
jwt.secret=dad-upeu-microservicios-jwt-secret-key-2026-super-segura-0123456789
jwt.expiration-ms=3600000
eureka.client.service-
url.defaultZone=${EUREKA_DEFAULT_ZONE:http://localhost:8761/eureka}
management.endpoints.web.exposure.include=health,info
```

Configuración ms-admin-api-gateway (`ms-admin-api-gateway.properties`)

```
server.port=8080
spring.cloud.gateway.discovery.locator.enabled=true
spring.cloud.gateway.discovery.locator.lower-case-service-id=true
spring.cloud.gateway.mvc.routes[0].id=auth-route
spring.cloud.gateway.mvc.routes[0].uri=lb://ms-auth
spring.cloud.gateway.mvc.routes[0].predicates[0]=Path=/api/auth/**
spring.cloud.gateway.mvc.routes[1].id=instructor-route
spring.cloud.gateway.mvc.routes[1].uri=lb://ms-gestion-instructor
spring.cloud.gateway.mvc.routes[1].predicates[0]=Path=/api/instructores/**
spring.cloud.gateway.mvc.routes[2].id=alumno-route
spring.cloud.gateway.mvc.routes[2].uri=lb://ms-gestion-alumno
spring.cloud.gateway.mvc.routes[2].predicates[0]=Path=/api/alumnos/**
spring.cloud.gateway.mvc.routes[3].id=taller-route
spring.cloud.gateway.mvc.routes[3].uri=lb://ms-gestion-taller
spring.cloud.gateway.mvc.routes[3].predicates[0]=Path=/api/talleres/**
```

Variables de Entorno Configurables

Variable	Valor por defecto	Descripción
<code>`EUREKA_DEFAULT_ZONE`</code>	<code>`http://localhost:8761/eureka`</code>	URL del Eureka Server
<code>`DB_HOST`</code>	<code>`localhost`</code>	Host de la base de datos
<code>`DB_PORT`</code>	<code>`5438/5439/5440/5441`</code>	Puerto de la base de datos
<code>`DB_NAME`</code>	<code>`instructor_db/alumno_db/taller_db/authdb`</code>	Nombre de la base de datos
<code>`SPRING_CONFIG_IMPORT`</code>	<code>`configserver:http://config-server:8888`</code>	URL del Config Server (Docker)

> **Nota:** Para entornos de producción, se recomienda migrar el Config Server a un repositorio Git remoto con perfiles separados (dev, test, prod) usando `spring.profiles.active`.

6. SCRIPTS DOCKERFILE Y DOCKER COMPOSE

Dockerfile — Patrón Multi-Stage (todos los microservicios)

```
# Etapa 1: Build
FROM eclipse-temurin:21-jdk-alpine AS build
```

```

WORKDIR /app
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN chmod +x ./mvnw && ./mvnw dependency:go-offline -B
COPY src ./src
RUN ./mvnw clean package -DskipTests -B

# Etapa 2: Runtime
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring
COPY --from=build /app/target/*.jar app.jar
EXPOSE <puerto-del-servicio>
ENTRYPOINT ["java", "-jar", "app.jar"]

```

Ventajas del Multi-Stage Build:

- Reduce el tamaño final de la imagen (~150MB Alpine vs ~500MB estándar)
- Imagen de runtime sin herramientas de compilación (más segura)
- Usuario no-root `spring:spring` para mayor seguridad

Docker Compose (`docker-compose.yml`)

```

# Fragmento representativo del servicio ms-auth
ms-auth:
  build: ./ms-auth
  ports:
    - "8084:8084"
  environment:
    SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-
server:8761/eureka
  depends_on:
    config-server:
      condition: service_healthy
    postgres-auth:
      condition: service_healthy
  networks: [ms-net]
  restart: on-failure

```

Características del docker-compose.yml:

- **Healthchecks:** Las bases de datos verifican su estado antes de que los servicios dependan de ellas
- **Dependencias ordenadas:** `depends\_on` + `condition: service\_healthy`
- **Red aislada:** Todos los servicios en la red `ms-net`
- **Volúmenes persistentes:** Datos de PostgreSQL entre reinicios
- **Restart policy:** `on-failure` para alta disponibilidad

Orden de Inicio Automático

1. Bases de datos PostgreSQL (con healthcheck)
2. Config Server (espera a que las DBs estén listas)
3. Eureka Server (espera a Config Server)
4. API Gateway (espera a Config + Eureka)
5. ms-auth (espera a Config + su DB)
6. ms-gestion-instructor, ms-gestion-alumno (esperan a Config + sus DBs)
7. ms-gestion-taller (espera a Config + su DB + instructor + alumno)

7. PRUEBAS UNITARIAS E INTEGRACIÓN

Pruebas Unitarias (JUnit)

Cada microservicio incluye pruebas de carga de contexto Spring:

Archivo de Prueba	Microservicio
`ConfigServerApplicationTests.java`	config-server
`EurekaServerApplicationTests.java`	eureka-server
`ApiGatewayApplicationTests.java`	api-gateway
`MsAuthApplicationTests.java`	ms-auth
`MsGestionInstructorApplicationTests.java`	ms-gestion-instructor
`MsGestionAlumnoApplicationTests.java`	ms-gestion-alumno
`MsGestionTallerApplicationTests.java`	ms-gestion-taller

Ejecutar pruebas unitarias:

```
# Un servicio específico
cd ms-auth && ./mvnw test

# Todos los servicios
./mvnw clean test
```

Pruebas de Integración E2E (Postman)

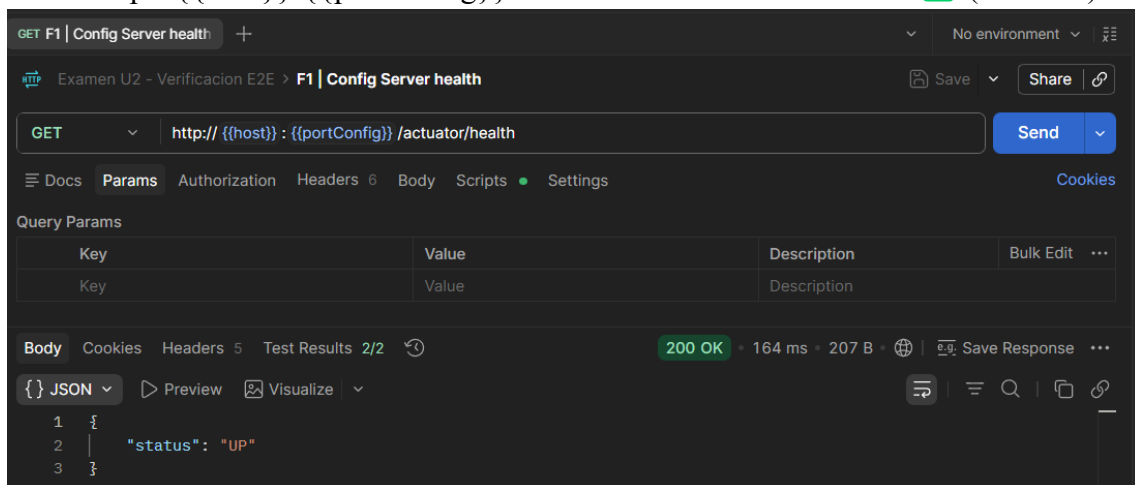
La colección DAD_Verificacion_E2E.postman_collection.json contiene **41 pruebas automatizadas**:

Fase	Descripción	Tests
F1	Config Server: health y configuración de servicios	3
F2	Eureka Server: registro de servicios (UP status)	1
F3	CRUD Instructores y Alumnos (validaciones, duplicados)	7
F4	Gateway routes y listado de talleres	2
F5	Feign: asignar instructor, inscribir alumno, detalle completo	3
F6	Circuit Breakers y health endpoints	1
F7	Balanceo de carga	1
F8	Auth: login (ADMIN/INSTRUCTOR/ALUMNO), registro, validación JWT	7
F9	Autorización por roles (403 Forbidden, 401 Unauthorized)	4
F10	Saga pattern: matrícula con compensación automática	6
Total		**41**

Evidencia de Pruebas Postman

F1 — Config Server Health

- **GET** `http://{{host}}:{{portConfig}}/ms-auth/default` → `200 OK`  (2/2 tests)



GET F1 | Config Server health GET F1 | Config serve ms-auth + No environment

Examen U2 - Verificación E2E > F1 | Config serve ms-auth Save Share

GET http://{{host}}:{{portConfig}}/ms-auth/default Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 2/2 200 OK 32 ms 1.18 KB Save Response

{ JSON Preview Visualize

```
1 {
2   "name": "ms-auth",
3   "profiles": [
4     "default"
5   ],
6   "label": null,
7   "version": null,
8   "state": null,
9   "propertySources": [
10    {
11      "name": "file:config-repo/ms-auth.properties",
12      "source": {
13        "server.port": "8084",
14        "spring.application.name": "ms-auth",
15        "spring.datasource.url": "jdbc:postgresql://localhost:5441/authdb",
16        "spring.datasource.username": "postgres",
17        "spring.datasource.password": "postgres",
18        "spring.jpa.hibernate.ddl-auto": "update",
19        "spring.jpa.show-sql": "true",
20        "jwt.secret": "dad-upeu-microservicios-jwt-secret-key-2026-super-segura-0123456789",
21        "jwt.expiration-ms": "3600000",
22        "eureka.client.service-url.defaultZone": "${EUREKA_DEFAULT_ZONE:http://localhost:8761/eureka}",
23        "management.endpoints.web.exposure.include": "health,info"
24      }
25    }
26  ]
27 }
```

- GET `http://{{host}}:{{portConfig}}/ms-admin-api-gateway/default` → `200 OK` ✓
(3/3 tests)

GET F1 | Config Server health GET F1 | Config serve ms-auth GET F1 | Config serve gateway + No environment

Examen U2 - Verificación E2E > F1 | Config serve gateway Save Share

GET http://{{host}}:{{portConfig}}/ms-admin-api-gateway/default Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 3/3 200 OK 30 ms 2.03 KB Save Response

{ JSON Preview Visualize

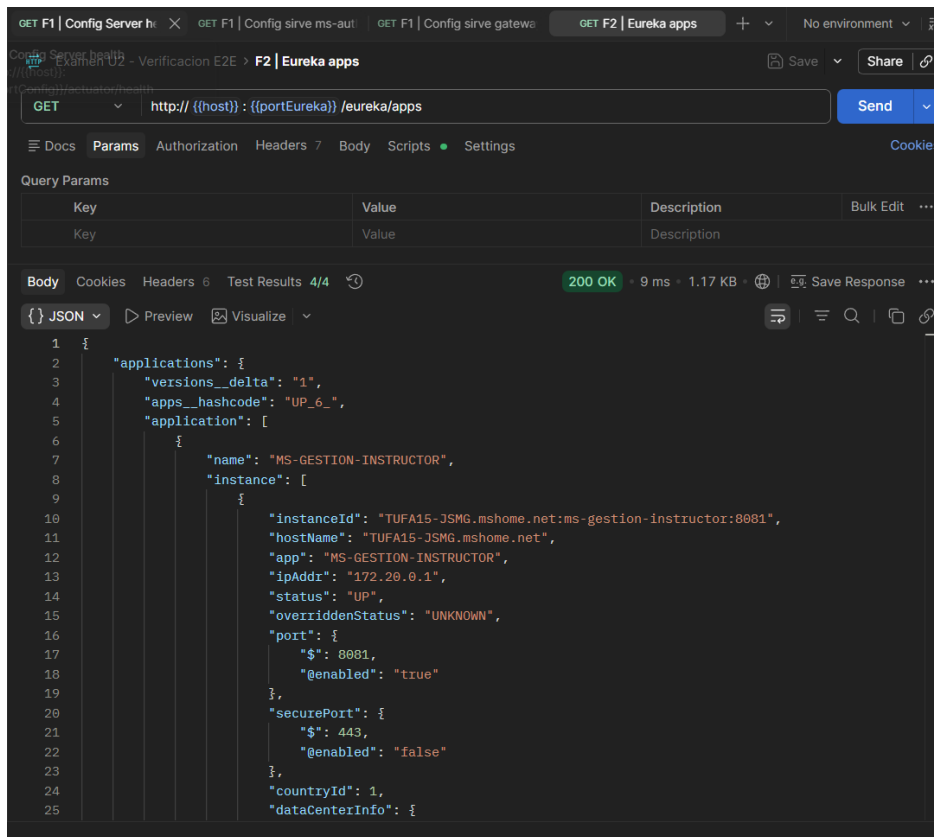
```

1 {
2   "name": "ms-admin-api-gateway",
3   "profiles": [
4     "default"
5   ],
6   "label": null,
7   "version": null,
8   "state": null,
9   "propertySources": [
10    {
11      "name": "file:config-repo/ms-admin-api-gateway.properties",
12      "source": {
13        "server.port": "8080",
14        "spring.cloud.gateway.discovery.locator.enabled": "true",
15        "spring.cloud.gateway.discovery.locator.lower-case-service-id": "true",
16        "spring.cloud.gateway.mvc.routes[0].id": "auth-route",
17        "spring.cloud.gateway.mvc.routes[0].uri": "lb://ms-auth",
18        "spring.cloud.gateway.mvc.routes[0].predicates[0]": "Path=/api/auth/**",
19        "spring.cloud.gateway.mvc.routes[1].id": "instructor-route",
20        "spring.cloud.gateway.mvc.routes[1].uri": "lb://ms-gestion-instructor",
21        "spring.cloud.gateway.mvc.routes[1].predicates[0]": "Path=/api/instructores/**",
22        "spring.cloud.gateway.mvc.routes[2].id": "alumno-route",
23        "spring.cloud.gateway.mvc.routes[2].uri": "lb://ms-gestion-alumno",
24        "spring.cloud.gateway.mvc.routes[2].predicates[0]": "Path=/api/alumnos/**",
25        "spring.cloud.gateway.mvc.routes[3].id": "taller-route"

```

F2 — Eureka Server

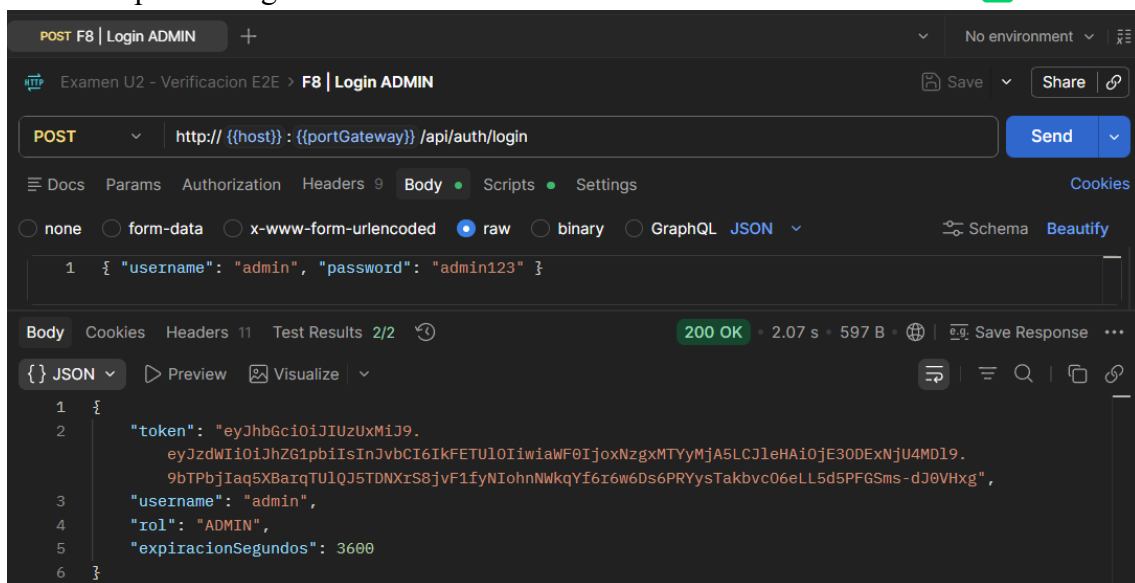
- GET `http://{{host}}:{{portEureka}}/eureka/apps` → `200 OK`  (4/4 tests)



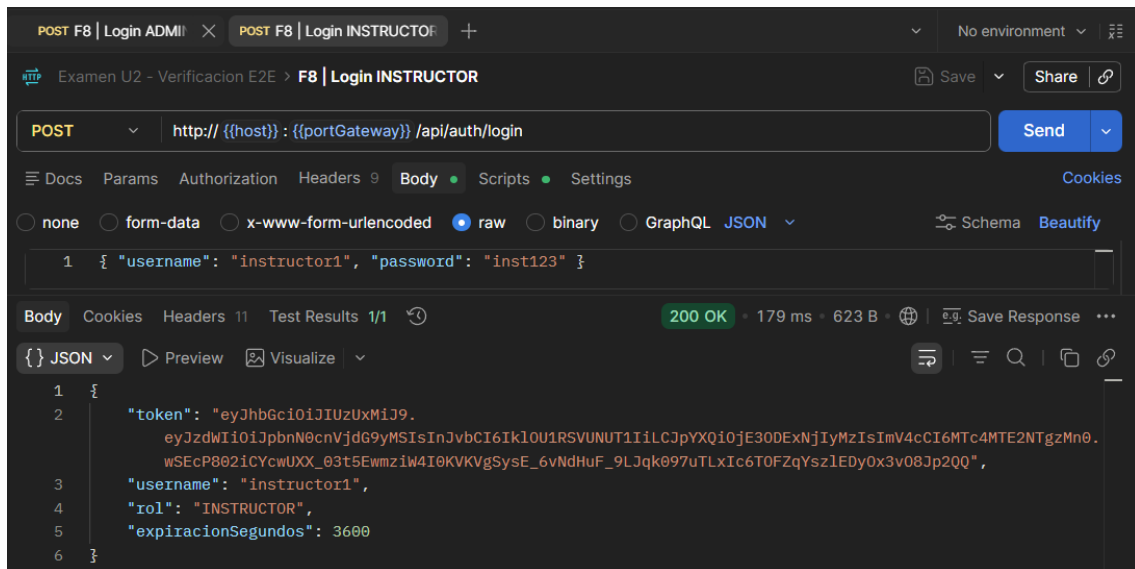
- 'MS-GESTION-INSTRUCTOR' registrado con status 'UP'

F8 — Autenticación JWT

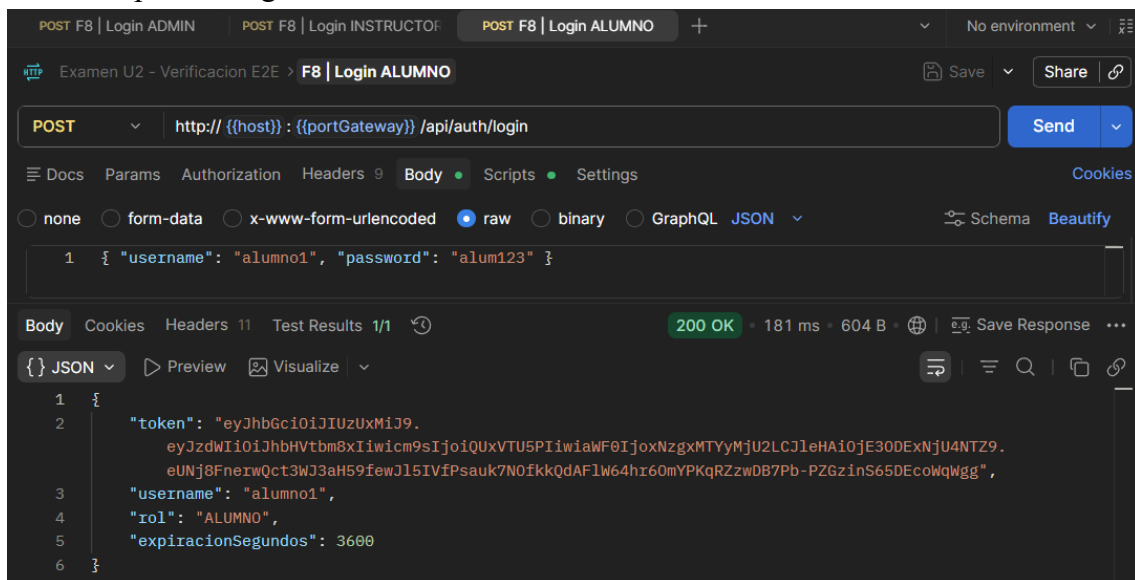
- POST '/api/auth/login' con 'admin/admin123' → '200 OK' + JWT token



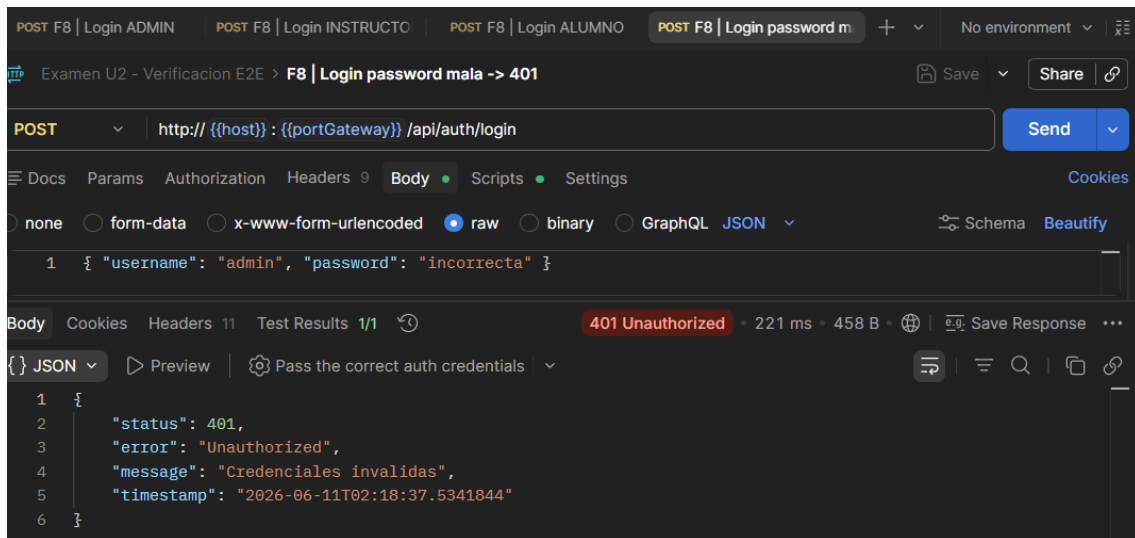
- POST '/api/auth/login' con 'instructor1/inst123' → '200 OK' + JWT token



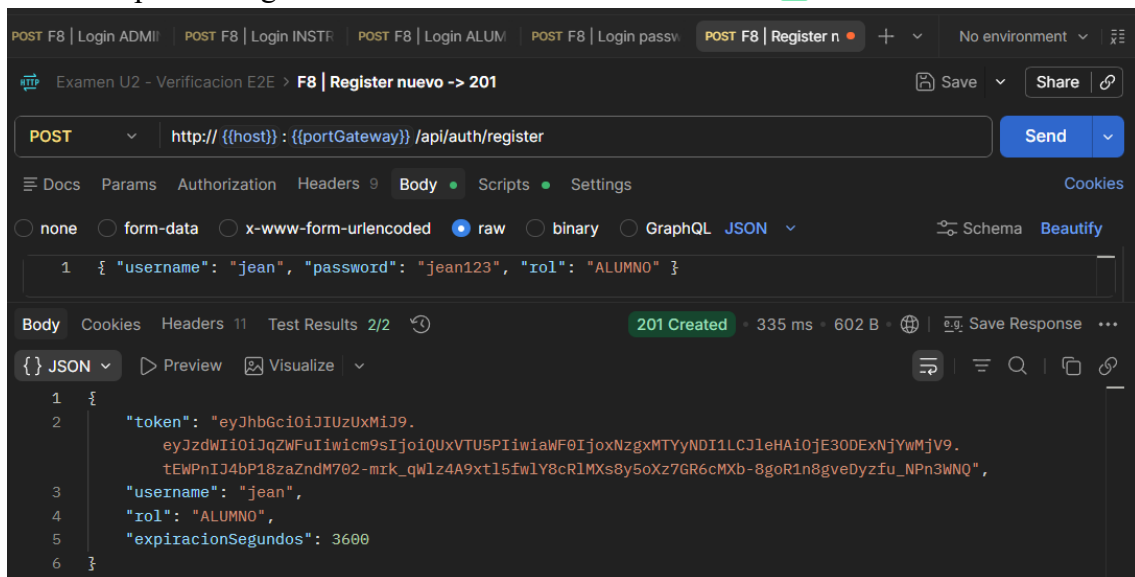
- **POST** `/api/auth/login` con `'alumno1/alum123'` → `'200 OK'` + JWT token



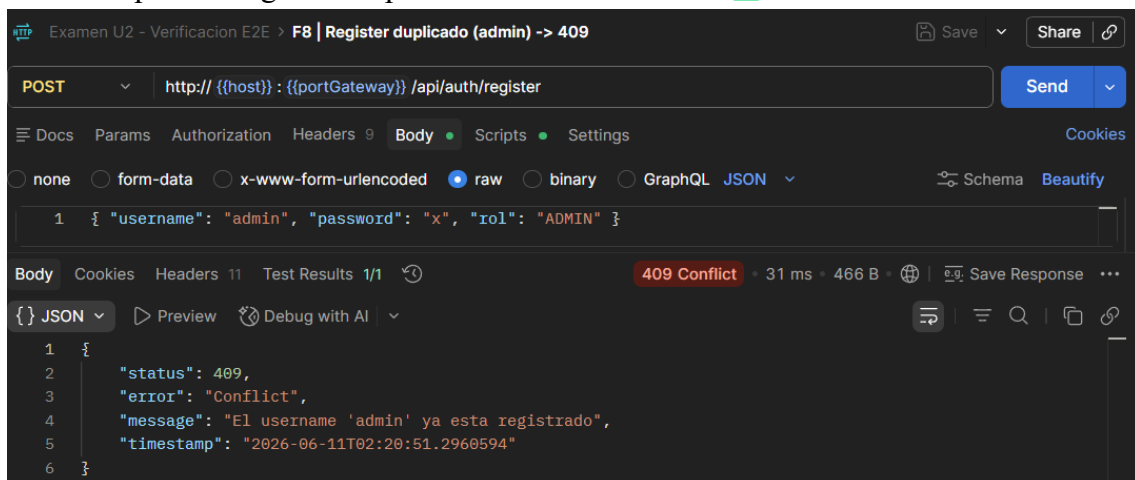
- **POST** `/api/auth/login` con password incorrecta → `'401 Unauthorized'`



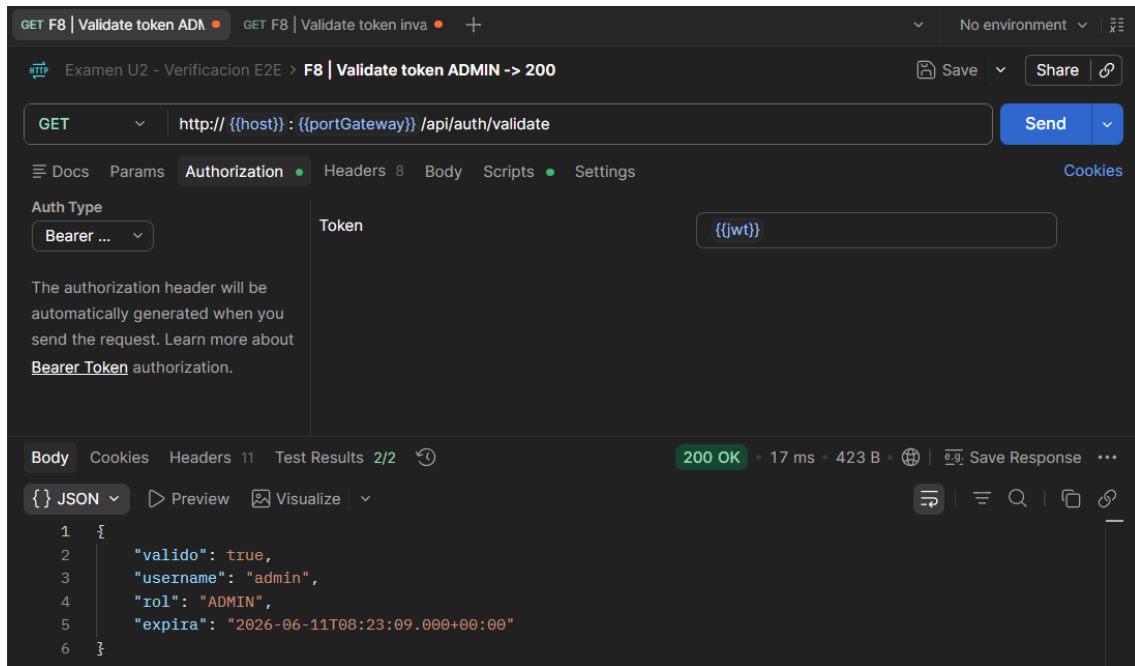
- `POST '/api/auth/register'` nuevo usuario → `'201 Created'` ✓



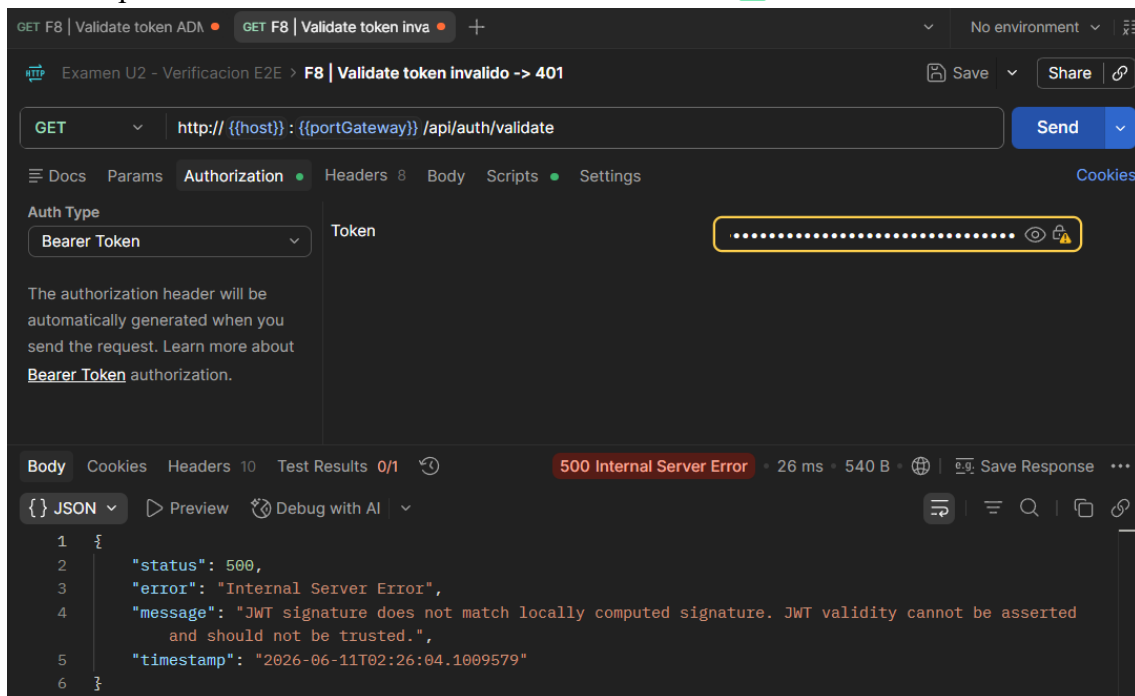
- `POST '/api/auth/register'` duplicado → `'409 Conflict'` ✓



- GET `/api/auth/validate` token válido → `200 OK` `{valido: true}` ✓

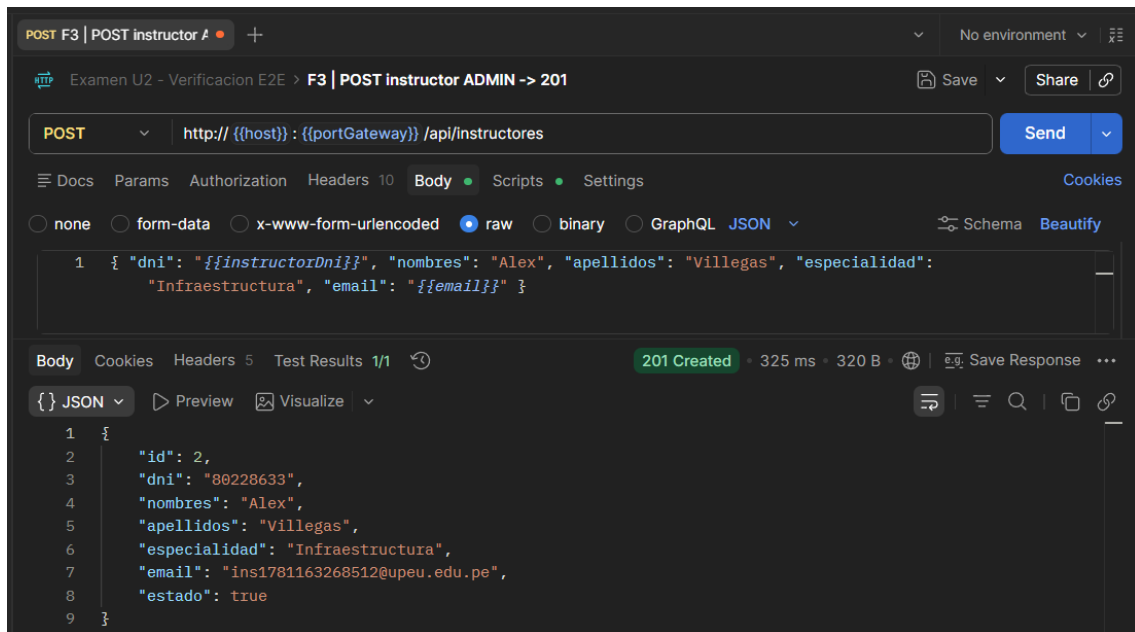


- GET `/api/auth/validate` token inválido → `500 / 401` ✓

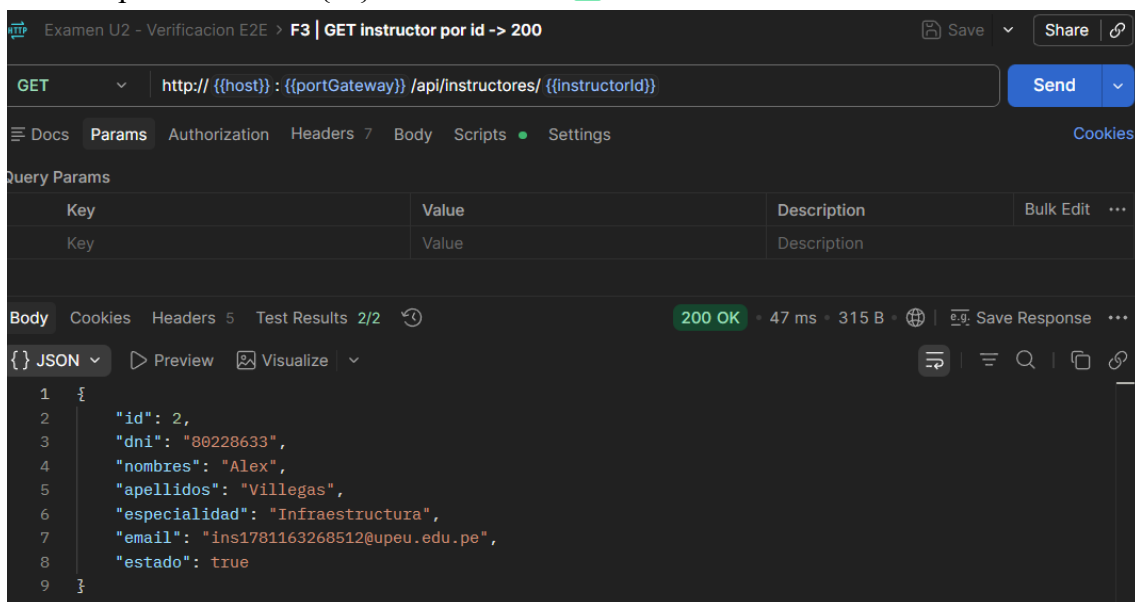


F3 — CRUD

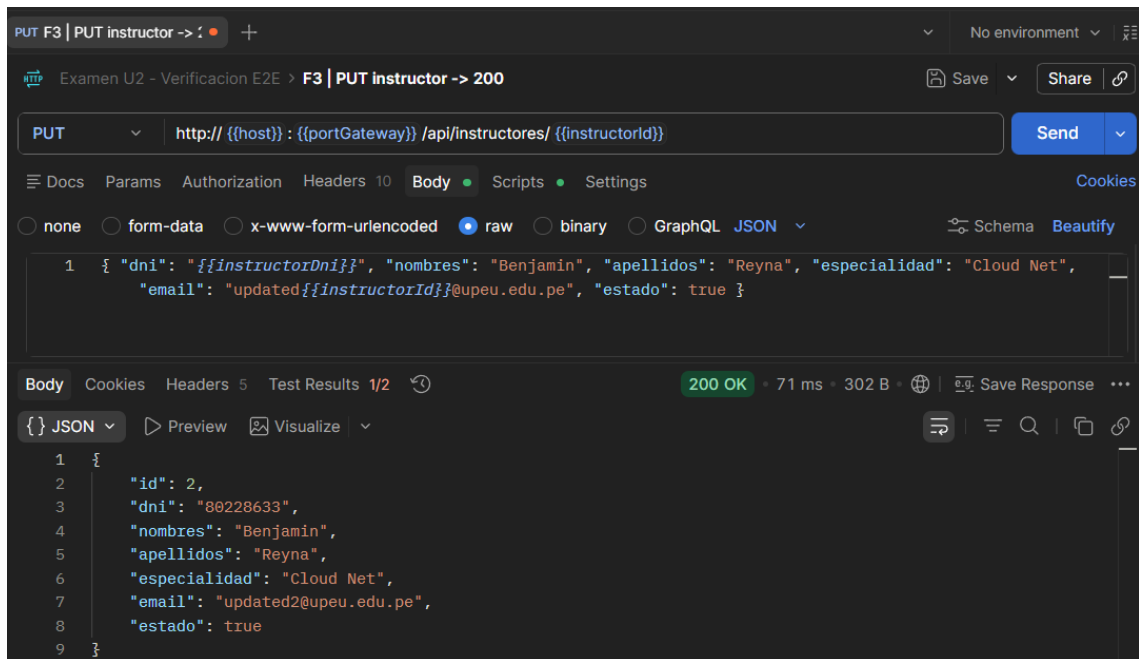
- POST `/api/instructores` (ADMIN) → `201 Created` `{id:2, nombres:"Alex", apellidos:"Villegas"}` ✓



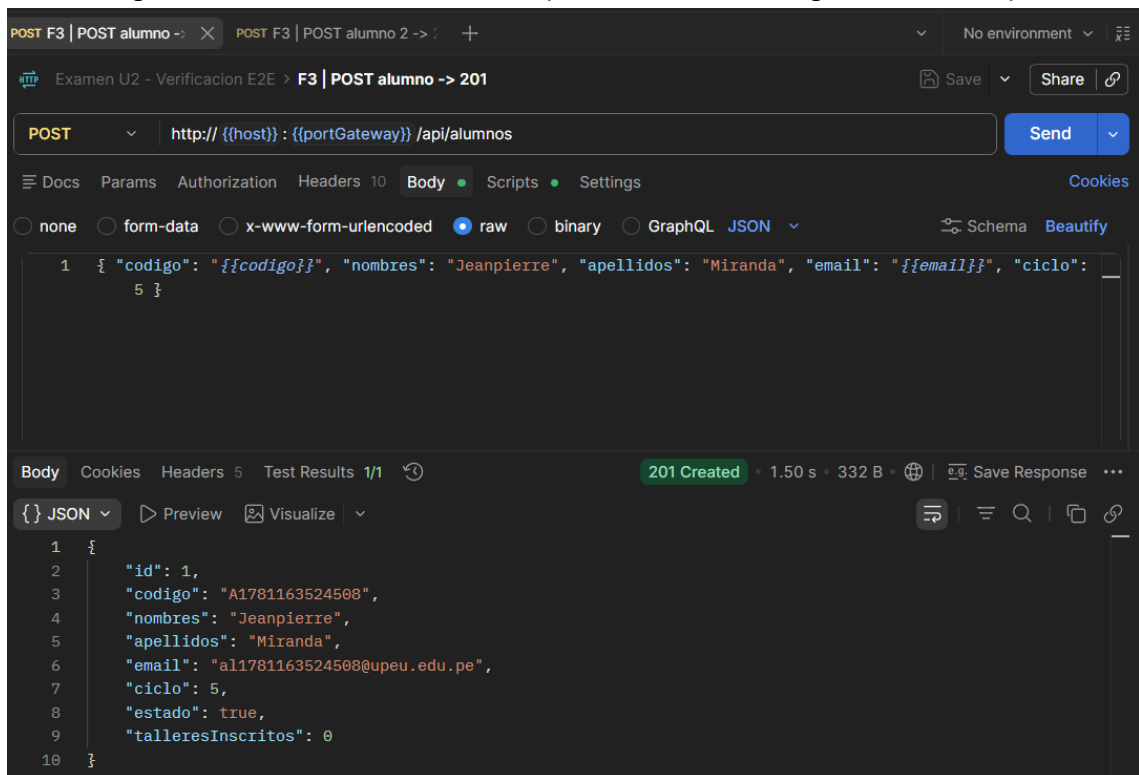
- GET `/api/instructores/{id}` → `200 OK` ✓

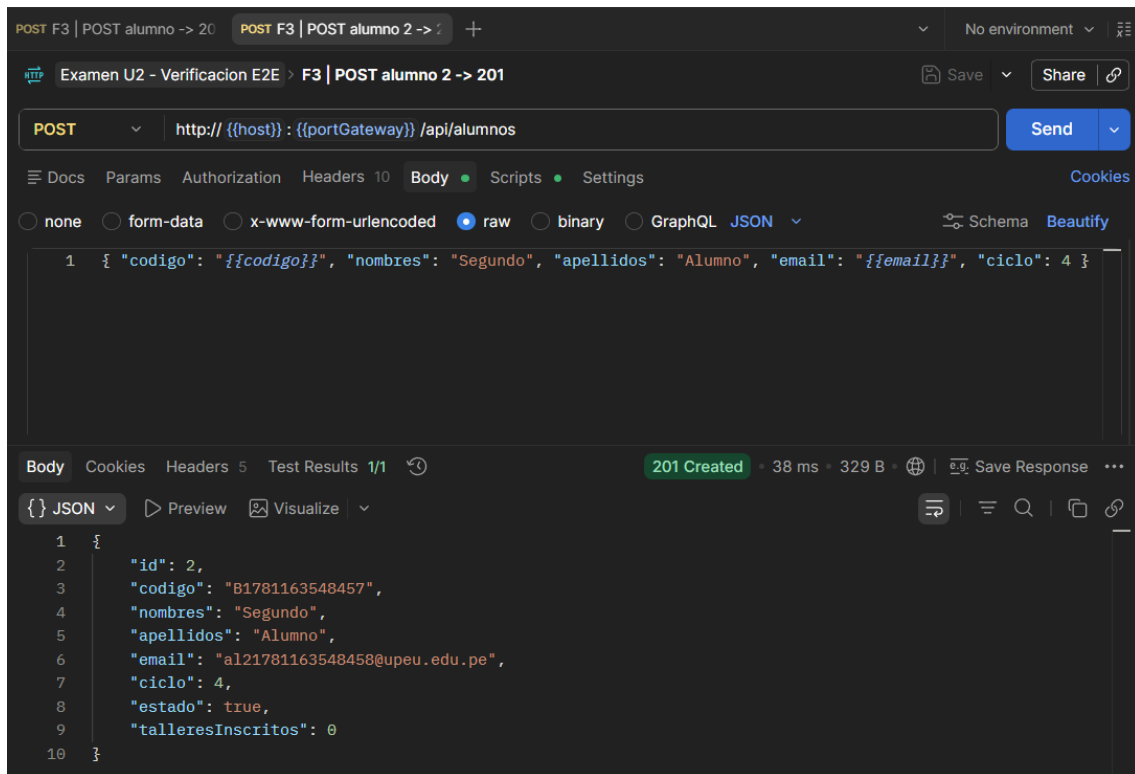


- PUT `/api/instructores/{id}` → `200 OK` `{ nombres:"Benjamin", especialidad:"Cloud Net" }` ✓

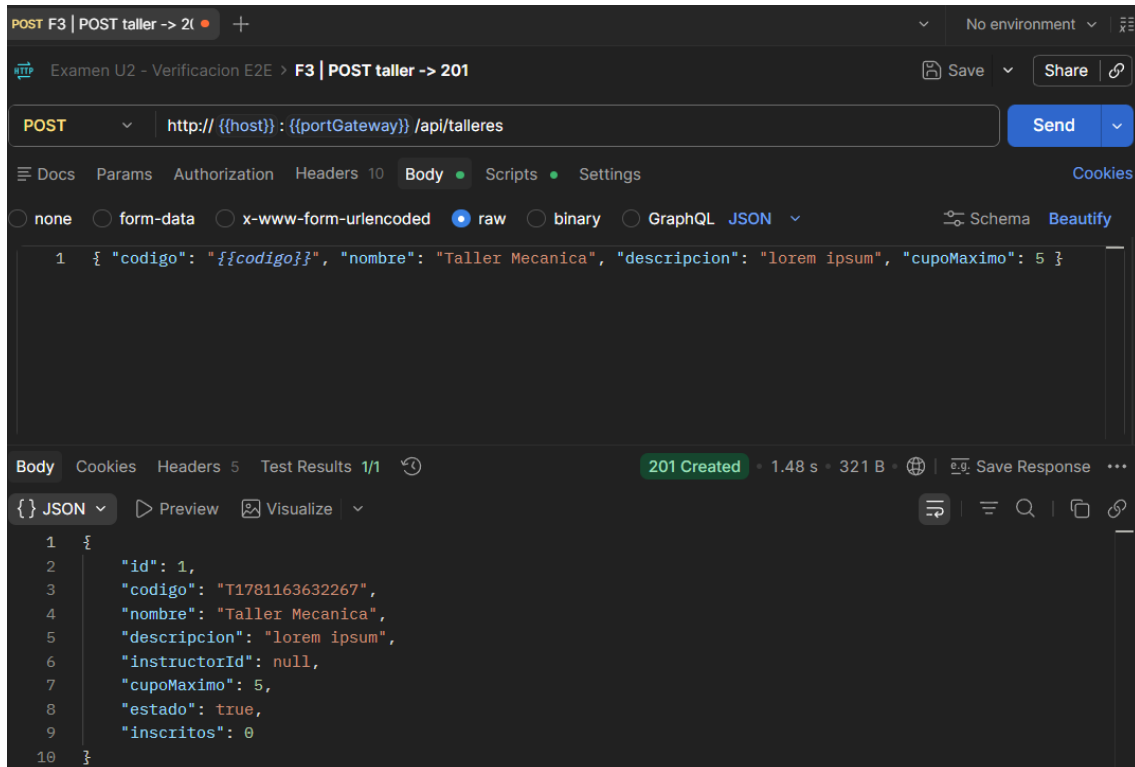


- `POST '/api/alumnos' → '201 Created' '{id:1, nombres:"Jeanpierre", ciclo:5}'` ✓





- `POST '/api/talleres' → '201 Created' '{id:1, nombre:"Taller Mecanica", cupoMaximo:5}'`



F5 — Feign (Comunicación entre microservicios)

- `POST '/api/talleres/{id}/asignar-instructor/{id}' → '200 OK' '{instructorId:2}'` ✓

The screenshot shows a Postman interface for a POST request. The URL is `http://{{host}}:{{portGateway}}/api/talleres/{{tallerId}}/asignar-instructor/{{instructorId}}`. The response status is **200 OK** with a response time of 436 ms and a body size of 312 B. The response body is a JSON object:

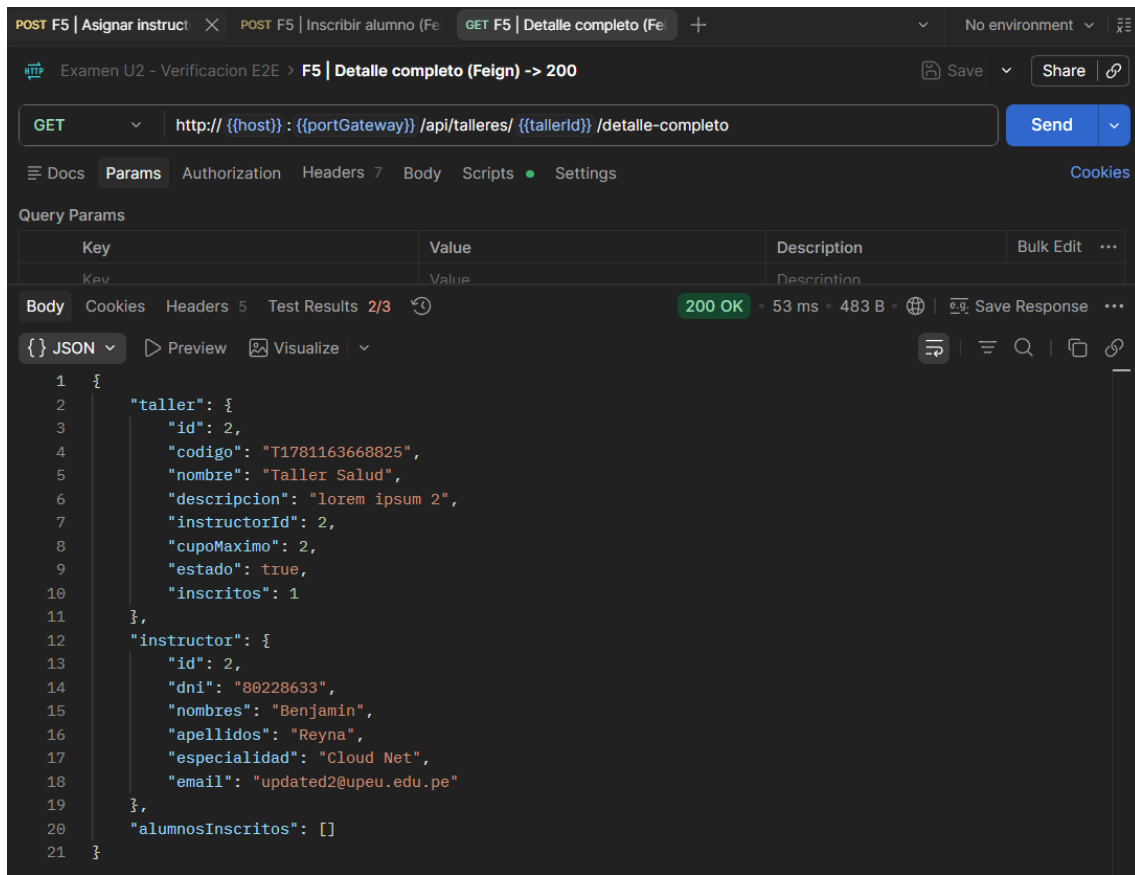
```
{
  "id": 2,
  "codigo": "T1781163668825",
  "nombre": "Taller Salud",
  "descripcion": "lorem ipsum 2",
  "instructorId": 2,
  "cupoMaximo": 2,
  "estado": true,
  "inscritos": 0
}
```

- `POST '/api/talleres/{id}/inscribir-alumno/{id}' → '201 Created' '{inscritos:1}'` ✓

The screenshot shows a Postman interface for a POST request. The URL is `http://{{host}}:{{portGateway}}/api/talleres/{{tallerId}}/inscribir-alumno/{{alumnoId}}`. The response status is **201 Created** with a response time of 182 ms and a body size of 317 B. The response body is a JSON object:

```
{
  "id": 2,
  "codigo": "T1781163668825",
  "nombre": "Taller Salud",
  "descripcion": "lorem ipsum 2",
  "instructorId": 2,
  "cupoMaximo": 2,
  "estado": true,
  "inscritos": 1
}
```

- `GET '/api/talleres/{id}/detalle-completo' → '200 OK' con 'instructor' + 'alumnosInscritos'` ✓



8. GUÍA DE INSTALACIÓN Y EJECUCIÓN

Prerrequisitos

- Java 21 o superior
- Docker Desktop instalado y corriendo
- Maven (o usar el wrapper `./mvnw`)
- Git

Opción A: Docker Compose (Recomendada)

Levanta **todos** los servicios (bases de datos + microservicios) automáticamente.

```
# 1. Clonar el repositorio
git clone https://github.com/JeanpierreSam/DAD-examen-u2.git
cd DAD-examen-u2
```

```
# 2. Construir las imágenes Docker
docker-compose build
```

```
# 3. Levantar todos los servicios en segundo plano
```



```
docker-compose up -d
```

4. Verificar que todos los contenedores están corriendo

```
docker-compose ps
```

5. Ver logs de un servicio específico

```
docker-compose logs -f ms-auth
```

6. Detener todos los servicios

```
docker-compose down
```

Opción B: Ejecución Local con IntelliJ

Ideal para desarrollo y debugging. Las bases de datos corren en Docker, los microservicios desde IntelliJ.

Paso 1: Levantar solo las bases de datos

```
docker-compose up -d postgres-auth postgres-instructor postgres-alumno postgres-taller
```

Paso 2: Verificar los 4 contenedores

```
docker ps --filter "name=postgres"
```

Paso 3: Ejecutar en IntelliJ (en orden estricto):

8. `ConfigServerApplication` (8888)
9. `EurekaServerApplication` (8761)
10. `MsAuthApplication` (8084)
11. `ApiGatewayApplication` (8080)
12. `MsGestionInstructorApplication` (8081)
13. `MsGestionAlumnoApplication` (8082)
14. `MsGestionTallerApplication` (8083)

Flujo de Autenticación JWT

Cliente → POST /api/auth/login → ms-auth → JWT token

Cliente → [cualquier ruta] + Header: Authorization: Bearer <token>

→ API Gateway AuthenticationGlobalFilter:

1. ¿Ruta pública (/api/auth/**)? → Pasar directo
2. ¿Header Authorization presente? → NO: 401
3. ¿Token válido? (JwtValidator) → NO: 401
4. ¿Rol autorizado para la ruta?
 - ADMIN: acceso total
 - GET: cualquier autenticado

- POST inscribir/matricular: solo ALUMNO
 - Otros POST/PUT/DELETE sin ser ADMIN: 403
5. Inyectar X-Auth-User y X-Auth-Rol al microservicio destino

Usuarios por Defecto (DataInitializer en ms-auth)

Username	Password	Rol	Acceso
`admin`	`admin123`	ADMIN	Total
`instructor1`	`inst123`	INSTRUCTOR	Solo lectura
`alumno1`	`alum123`	ALUMNO	Lectura + inscripción

Políticas de Autorización por Rol

Rol	GET	POST/PUT/DELETE	Inscribir/Matricular
ADMIN	✓	✓	✓
INSTRUCTOR	✓	✗	✗
ALUMNO	✓	✗	✓

Proyecto desarrollado para fines académicos — Universidad Peruana Unión, Lima 2026